



DIPARTIMENTO DI ELETTRONICA, INFORMAZIONE E BIOINGEGNERIA



POLITECNICO
DI MILANO

Fondamenti di informatica



Rappresentazione della informazione – parte 2



- I numeri decimali compresi tra 0 e 1 possono essere rappresentati con la sequenza di bit

$$0, b_{-1} b_{-2} \dots b_{-n} \text{ essendo } b_i \in \{0, 1\}$$

che rappresenta in base 2 il valore:

$$b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \dots + b_{-n} \times 2^{-n}$$

- Esempio: $0,1011_{\text{bin}}$ (in binario)

$$\begin{aligned} 0,1011_{\text{bin}} &= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} = 1/2 + 1/8 + 1/16 = \\ &= 0,5 + 0,125 + 0,0625 = 0,6875_{\text{dec}} \end{aligned}$$

- Con n bit ($n \geq 1$) codifichiamo 2^n numeri nell'intervallo $[0; 1 - 2^{-n}]$
 - Avendo n cifre (un numero finito!) per rappresentare la parte decimale, ci sarà un errore di approssimazione minore di 2^{-n}



- Algoritmo per convertire un numero n decimale compreso tra 0 e 1 dalla base 10 alla base 2:
 1. Il risultato in base 2 è inizializzato a 0,
 2. Moltiplicare n per 2
 3. La parte intera del risultato viene aggiunta come cifra meno significativa del numero in base 2
 4. Si torna al passo 2 considerando la parte decimale del risultato al posto di n
- Terminazione dell'algoritmo
 - Soltanto i numeri del tipo $m/2^n$ (con n ed m interi) possono essere rappresentati con un numero finito di cifre
 - In alternativa ci si ferma quando il numero di cifre calcolate costituisce un'approssimazione sufficiente



- Esempio: convertire $(0.587)_{10}$ in base 2 usando 6 cifre decimali

$$0.587 \times 2 = 1.174 \text{ parte intera} = 1$$

$$0.174 \times 2 = 0.348 \text{ parte intera} = 0$$

$$0.348 \times 2 = 0.696 \text{ parte intera} = 0$$

$$0.696 \times 2 = 1.392 \text{ parte intera} = 1$$

$$0.392 \times 2 = 0.784 \text{ parte intera} = 0$$

$$0.784 \times 2 = 1.568 \text{ parte intera} = 1$$

- Il risultato (approssimato) è $0,100101_2$



- Un numero decimale è rappresentato in binario concatenando la parte intera con quella della parte frazionaria:

$b_{k-1} \dots b_2 b_1 b_0, b_{-1} b_{-2} \dots b_{-n}$ essendo $b_i \in \{0, 1\}$

che rappresenta in base 2 il valore:

$$b_{k-1} \times 2^{k-1} + \dots + b_2 \times 2^2 + b_1 \times 2^1 + b_0 \times 2^0 + b_{-1} \times 2^{-1} + b_{-2} \times 2^{-2} + \dots + b_{-n} \times 2^{-n}$$

- Esempio:

$$19,6875_{\text{dec}} = 10011,1011_{\text{bin}}$$

poiché si ha:

$$19_{\text{dec}} = 10011_{\text{bin}} \quad \text{e} \quad 0,6875_{\text{dec}} = 0,1011_{\text{bin}}$$

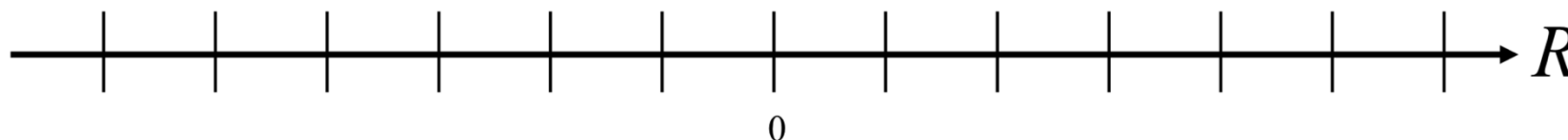
- Necessità: come rappresentiamo la virgola nel calcolatore?



- In un calcolatore si può rappresentare un numero decimale con la notazione in **virgola fissa** (o **fixed point**) concatenando la rappresentazione della parte intera (**in modulo e segno**) con quella della parte frazionaria
 - La rappresentazione utilizza un numero prefissato di bit per rappresentare la parte intera ed per quella frazionaria
 - La virgola è rappresentata implicitamente
 - Il segno è rappresentato mediante la notazione modulo e segno, utilizzando il primo bit a sinistra della parte intera
- Esempio: volendo codificare il numero $+19,6875_{\text{dec}}$ in notazione virgola fissa utilizzando 7 bit per la parte intera (compresa di segno) e 5 bit per la parte frazionaria otterremmo: 001001110110



- La codifica ha una precisione costante lungo l'asse reale:



- È un sistema di rappresentazione semplice, ma poco flessibile, e può condurre a sprechi di bit
 - Come dimensioniamo la parte intera e quella frazionaria?
- L'impatto dell'errore sulla precisione della rappresentazione aumenta con il decrescere del numero
- Le operazioni aritmetiche funzionano allo stesso modo dei numeri interi



- Per superare i limiti di rappresentazione in virgola fissa è stata introdotta la rappresentazione in virgola mobile
- La rappresentazione in **virgola mobile** (o **floating point**) è usata spesso in base 10 (si chiama **notazione scientifica**):
 $1,37 \times 10^7$ indica 13.700.000
si lascia una sola cifra diversa da 0 prima della virgola
- La rappresentazione binaria in virgola mobile funziona nello stesso modo

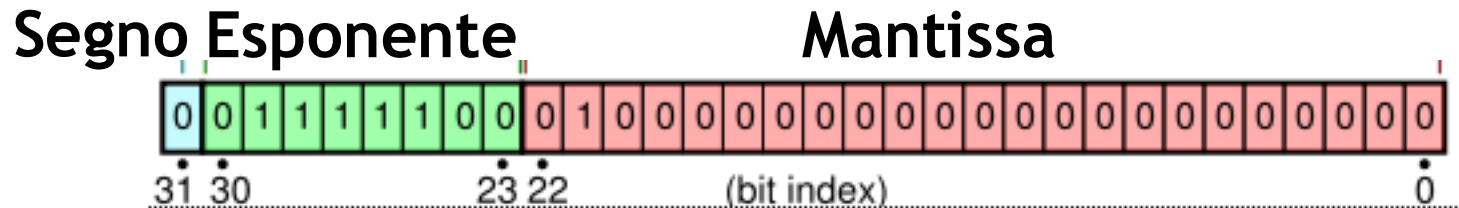
$$R_{\text{virgola mobile}} = \pm 1,M \times 2^E$$



- I numeri in virgola mobile sono rappresentati comunemente con lo standard IEEE 754
- Esistono due formati:
 - Singola precisione, che usa 32 bit
 - Segno: 1 bit
 - Esponente: 8 bit
 - Mantissa: 23 bit
 - Doppia precisione, che usa 64 bit
 - Segno: 1 bit
 - Esponente: 11 bit
 - Mantissa: 52 bit



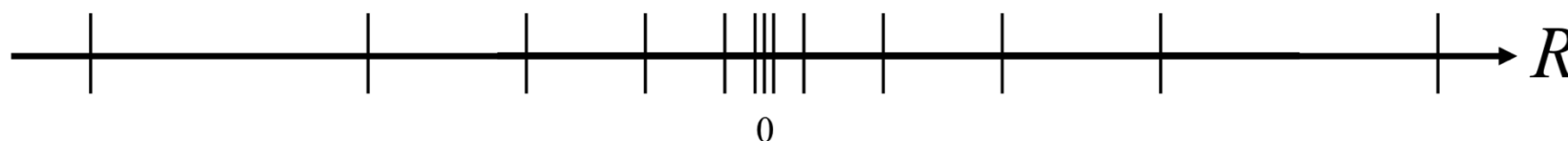
- Singola precisione:



- Segno: 1 per numero negativo, 0 per numero positivo
- All'esponente (in binario naturale) viene sommata la costante 127 (detta **eccesso**)
 - Esempio: $2^1 \rightarrow$ esponente=127+1=128
 - Esponenti < 127 rappresentano numeri (in modulo) nell'intervallo $[0;1)$
 - Esponenti ≥ 127 rappresentano numeri (in modulo) nell'intervallo $[1;\infty)$
 - Eccesso=1023 nel caso doppia precisione
- La mantissa è **normalizzata** cioè la virgola viene spostata per ottenere la forma 1,X Quindi si salva solo X
 - Tranne quando l'esponente è 0; in tal caso la mantissa è 0,X



- Se l'esponente è nell'intervallo $[1; 255)$
 - $\text{Valore}_2 = (-1)^{\text{segno}} \times 1, \text{mantissa} \times 2^{(\text{esponente}-127)}$
- Se l'esponente è 0
 - $\text{Valore}_2 = (-1)^{\text{segno}} \times 0, \text{mantissa} \times 2^{(-126)}$
- Valori particolari:
 - Esponente=255 e mantissa!=0 -> valore= NaN
 - Esponente=255, mantissa=0 e segno=0 -> valore= +Inf
 - Esponente=255, mantissa=0 e segno=1 -> valore = -Inf
 - Esponente=0, mantissa=0 e segno=0 -> valore= +0
 - Esponente=0, mantissa=0 e segno=1 -> valore= -0
- La precisione decresce col crescere del numero





- Si converte il numero decimale in binario virgola fissa
 - Calcolando separatamente la parte intera e la parte decimale e concatenandole
 - Esempio: $+19,6875_{\text{dec}} = +10011,1011_{\text{bin}}$
- Si normalizza il numero binario
 - Esempio: $+10011,1011 = +1,00111011 * 2^4$
- Si calcolano segno, esponente e mantissa
 - Esempio:
 - Segno = 0 (numero positivo)
 - Esponente = $4 + 127 = 131_{10} = 10000011_2$ (esponente ottenuto durante la normalizzazione sommato all'eccesso)
 - Mantissa = 001110110000000000000000 (parte decimale del numero normalizzato)
 - Risultato = 0 10000011 001110110000000000000000



- I caratteri sono rappresentati con una codifica numerica
- La più comune è la codifica ASCII (American Standard Code for Information Interchange) che utilizza 8bit

Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char
00000000	0	Null	00100000	32	Spc	01000000	64	@	01100000	96	`
00000001	1	Start of heading	00100001	33	!	01000001	65	A	01100001	97	a
00000010	2	Start of text	00100010	34	"	01000010	66	B	01100010	98	b
00000011	3	End of text	00100011	35	#	01000011	67	C	01100011	99	c
00000100	4	End of transmit	00100100	36	\$	01000100	68	D	01100100	100	d
00000101	5	Enquiry	00100101	37	%	01000101	69	E	01100101	101	e
00000110	6	Acknowledge	00100110	38	&	01000110	70	F	01100110	102	f
00000111	7	Audible bell	00100111	39	'	01000111	71	G	01100111	103	g
00001000	8	Backspace	00101000	40	(	01001000	72	H	01101000	104	h
00001001	9	Horizontal tab	00101001	41	)	01001001	73	I	01101001	105	i
00001010	10	Line feed	00101010	42	*	01001010	74	J	01101010	106	j
00001011	11	Vertical tab	00101011	43	+	01001011	75	K	01101011	107	k
00001100	12	Form Feed	00101100	44	,	01001100	76	L	01101100	108	l
00001101	13	Carriage return	00101101	45	-	01001101	77	M	01101101	109	m
00001110	14	Shift out	00101110	46	.	01001110	78	N	01101110	110	n
00001111	15	Shift in	00101111	47	/	01001111	79	O	01101111	111	o
00010000	16	Data link escape	00110000	48	0	01010000	80	P	01110000	112	p
00010001	17	Device control 1	00110001	49	1	01010001	81	Q	01110001	113	q
00010010	18	Device control 2	00110010	50	2	01010010	82	R	01110010	114	r
00010011	19	Device control 3	00110011	51	3	01010011	83	S	01110011	115	s
00010100	20	Device control 4	00110100	52	4	01010100	84	T	01110100	116	t
00010101	21	Neg. acknowledge	00110101	53	5	01010101	85	U	01110101	117	u
00010110	22	Synchronous idle	00110110	54	6	01010110	86	V	01110110	118	v
00010111	23	End trans. block	00110111	55	7	01010111	87	W	01110111	119	w
00011000	24	Cancel	00111000	56	8	01011000	88	X	01111000	120	x
00011001	25	End of medium	00111001	57	9	01011001	89	Y	01111001	121	y
00011010	26	Substitution	00111010	58	:	01011010	90	Z	01111010	122	z
00011011	27	Escape	00111011	59	;	01011011	91	[	01111011	123	{
00011100	28	File separator	00111100	60	<	01011100	92	\	01111100	124	
00011101	29	Group separator	00111101	61	=	01011101	93	]	01111101	125	}
00011110	30	Record Separator	00111110	62	>	01011110	94	^	01111110	126	~
00011111	31	Unit separator	00111111	63	?	01011111	95	_	01111111	127	Del



- Alle volte ci è più comodo scrivere in modo più compatto un dato binario (qualunque tipo di informazione esso rappresenti)
- Si usa spesso rappresentare in base 8 o 16 le informazioni binarie
- Base 8:
 - Alfabeto: $\{0, 1, 2, 3, 4, 5, 6, 7\}$
 - La conversione bin- $\rightarrow$ oct avviene suddividendo i bit del valore binario in gruppi da 3 cifre e codificando ciascuna sottosequenza con il simbolo corrispondente in base 8
 - Esempio: $100110_2 \rightarrow 100\ 110_2 \rightarrow 46_8$
 - La conversione oct- $\rightarrow$ bit avviene sostituendo ciascuna cifra ottale con la corrispondente codifica in binario utilizzando 3 bit per ciascuna cifra
 - Esempio: $23_8 \rightarrow 010\ 011_2$



- Base 16:
 - Alfabeto: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}
 - La conversione bin- $\rightarrow$ hex avviene suddividendo i bit del valore binario in gruppi da 4 cifre e codificando ciascuna sottosequenza con il simbolo corrispondente in esadecimale
 - Esempio: $10100110_2 \rightarrow 1010\ 0110_2 \rightarrow A6_{16}$
 - La conversione hex- $\rightarrow$ bit avviene sostituendo ciascuna cifra esadecimale con la corrispondente codifica in binario utilizzando 4 bit per ciascuna cifra
 - Esempio: $2C_{16} \rightarrow 0010\ 1100_2$
 - I numeri esadecimali vengono in genere indicati con un prefisso 0x
- ATTENZIONE: il numero di cifre del numero binario deve essere multiplo di 3 nel caso di conversione in base 8 e 4 nel caso di base 16
 - Se necessario il numero binario va esteso di segno (in accordi alla notazione binaria utilizzata) per poter arrivare alla lunghezza richiesta